

UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE COLOMBIA

Ingeniería de Sistemas y Computación

Programación 3

Ejercicio de práctica - Sockets (2 clientes y transformación de mensaje)

Versión

1.0

Propósito

Practicar el uso de **sockets TCP** y **hilos/sincronización** en Java para coordinar dos clientes:

1. El **Cliente 1** envía un mensaje de **texto en minúsculas**.
2. El **servidor** recibe ese mensaje, lo transforma a **mayúsculas**.
3. El **Cliente 2** recibe el resultado desde el servidor.

Enunciado

Crear un programa cliente-servidor en Java donde:

1. Existe **un solo servidor** que escucha en un puerto TCP.
2. El servidor atiende **mínimo dos clientes**:
 - Cliente 1: envía un mensaje.
 - Cliente 2: recibe el mensaje transformado.
3. El servidor debe:
 - Recibir del Cliente 1 un mensaje.
 - Validar que el mensaje esté en minúsculas.
 - Transformar a mayúsculas.
 - Enviar el mensaje transformado al Cliente 2.
4. El manejo debe garantizar que el mensaje enviado por el Cliente 1 se entregue al Cliente 2 (no se debe "perder" ni mezclar mensajes).

Entradas (parámetros)

Servidor: MsgServer

Ejecutar con: `java MsgServer <port>`

Ejemplo: `java MsgServer 5000`

Cliente 1: MsgClient1

Ejecutar con: `java MsgClient1 <host> <port> <mensaje>`

Ejemplo: `java MsgClient1 localhost 5000 "hola mundo"`

El `<mensaje>` debe enviarse en **minúsculas** (ver validación en Reglas).

Cliente 2: MsgClient2

Ejecutar con: `java MsgClient2 <host> <port>`

Ejemplo: `java MsgClient2 localhost 5000`

Protocolo de comunicación (obligatorio)

Usar `DataInputStream` y `DataOutputStream`.

Cliente 1 → Servidor

1. Envía `int msgLen` (longitud del mensaje en bytes UTF-8)
2. Envía `byte[msgLen]` con el mensaje en UTF-8

Servidor → Cliente 2

1. Envía `boolean ok`
2. Envía `int msgLen` (longitud del mensaje en bytes UTF-8)
3. Envía `byte[msgLen]` con el mensaje transformado en UTF-8

Validación

Si el mensaje enviado por el Cliente 1 **no** está en minúsculas, el servidor debe mandar `ok=false` .

Reglas (obligatorias)

1. El Cliente 1 envía un mensaje de texto.
2. El servidor valida que el mensaje cumpla:
 - `mensaje.equals(mensaje.toLowerCase(Locale.ROOT))`
3. Transformación:

- Convertir el mensaje a mayúsculas con `toUpperCase(Locale.ROOT)` .

4. Coordinación:

- El servidor debe sincronizar el flujo para asegurar que el mensaje recibido del Cliente 1 se envíe al Cliente 2.
- Se permite usar un hilo para cada cliente y una variable compartida protegida, o usar `wait/notify` .

Requerimientos funcionales

1. El servidor debe:

- Crear un `ServerSocket` .
- Aceptar conexión del Cliente 1 y leer el mensaje.
- Aceptar conexión del Cliente 2 y enviar el resultado transformado.

2. El Cliente 1 debe:

- Enviar el mensaje usando el protocolo.

3. El Cliente 2 debe:

- Leer `ok` y el mensaje final.
- Mostrar en consola lo recibido.

4. Manejo de errores mínimo:

- Si ocurre una desconexión abrupta, el servidor no debe finalizar inesperadamente.

Criterios de aceptación

1. Si el Cliente 1 envía `hola mundo` , el Cliente 2 recibe `HOLA MUNDO` .
2. Si el Cliente 1 envía un texto que no está en minúsculas (por ejemplo `Hola`), el Cliente 2 recibe `ok=false` .
3. El servidor coordina correctamente: el resultado del Cliente 1 se entrega al Cliente 2.

Entregables

1. Código fuente en Java:

- `MsgServer.java`
- `MsgClient1.java` // recibe
- `MsgClient2.java` // envía